

ROS2 Doctor

BRIEF

1. Domain

Robot Field Operations — Post-run Diagnostics trên nền tảng ROS2, tập trung vào AMR (Autonomous Mobile Robot) sử dụng Nav2 stack.

Đây không phải domain phát triển thuật toán robot (perception, SLAM, planning).

Đây là domain vận hành robot sau khi đã triển khai thực tế — nơi kỹ sư phải đối mặt với câu hỏi hàng ngày: "Robot vừa chạy xong ca, nó bị lỗi/hành vi lạ — chuyện gì đã xảy ra?"

Câu trả lời nằm trong rosbag — file ghi lại toàn bộ dữ liệu ROS2 trong suốt quá trình robot hoạt động. Nhưng đọc và phân tích rosbag là công việc thủ công, chậm, lặp đi lặp lại, và phụ thuộc nặng vào kinh nghiệm cá nhân.

2. Người dùng mục tiêu

Kỹ sư robotics đang vận hành AMR thực tế trong:

- Nhà kho, nhà máy (logistics AMR)
- Giao hàng ngoài trời (delivery robot)
- Xe tự hành giai đoạn testing
- Robot nông nghiệp, robot khảo sát

Đặc điểm chung: mỗi ngày đều phải debug robot qua rosbag, và đây là phần tốn thời gian nhất trong workflow.

3. Pain Points

3.1. Tốn thời gian — "Mở bag ra rồi không biết nhìn đâu trước"

Một rosbag điển hình chứa 20–50 topic, hàng triệu message, thời lượng 10 phút đến vài giờ. Kỹ sư phải mở từng tool riêng biệt (rqt_bag, rviz, plotjuggler, terminal) để xem từng khía cạnh. Không có cách nào nhìn tổng quan "bag này có gì bất thường" trong một cái nhìn duy nhất. Thời gian trung bình: 1–4 giờ cho MỘT bag file.

3.2. Phụ thuộc kinh nghiệm — "Junior không biết cái gì là bất thường"

Nhìn vào frequency plot của /camera/image_raw thấy tụt từ 30Hz xuống 12Hz — senior engineer biết ngay đó là dấu hiệu CPU overload. Nhưng junior engineer không biết 30Hz là frequency kỳ vọng, không biết 12Hz là "tụt" hay "bình thường", không biết nguyên nhân có thể là gì. Kiến thức debug này thường là tribal knowledge — truyền miệng trong team, không được document hóa.

3.3. Không có giải thích & đề xuất — "Biết lỗi rồi, giờ làm gì?"

Ngay cả khi đã xác định được "TF giữa odom → base_link bị gãy ở giây 60", câu hỏi tiếp theo là: Node nào chịu trách nhiệm? Node đó crash hay chỉ dừng publish? Nguyên nhân gốc là gì? Fix bằng cách nào? Hiện tại không có tool nào trả lời chuỗi câu hỏi này tự động.

3.4. Công cụ rời rạc — "Phải mở 4–5 tool khác nhau"

ros2 bag info (terminal), rqt_bag/plotjuggler (GUI), rviz2 (3D), grep log (terminal), ghi chú (notepad) — không có nơi nào tổng hợp tất cả thành một bức tranh duy nhất. Kỹ sư phải tự ghép nối thông tin trong đầu.

4. Giải pháp đề xuất

Xây dựng một agent phân tích file rosbag, tự động phát hiện bất thường (topic mất, drop rate cao, lệch thời gian, cây TF không nhất quán), giải thích khả năng nguyên nhân bằng ngôn ngữ tự nhiên và đề xuất hướng khắc phục, kèm trực quan timeline các topic.

Agent chỉ chẩn đoán và gợi ý, kỹ sư quyết định và duyệt hành động khắc phục (human-in-the-loop); phân tích trên log ghi sẵn, không tự động can thiệp hệ thống robot đang chạy.

5. Phạm vi (Scope)

Tập trung vào AMR chạy Nav2 stack. Không cố gắng cover tất cả loại robot (manipulation arm, drone, xe tự hành) vì mỗi loại cần domain knowledge riêng, analyzer riêng, prompt riêng.

Trong phạm vi	Ngoài phạm vi
AMR chạy Nav2 stack	Robot cánh tay (manipulation)
Phân tích rosbag post-run	Monitoring real-time robot đang chạy
6 loại lỗi phổ biến Nav2	Lỗi đặc thù perception/SLAM
Chẩn đoán + đề xuất	Tự động fix/can thiệp hệ thống

6. Tech Stack

Thành phần	Công nghệ	Lý do chọn
Parse bag	rosbag2_py, mcap	API chuẩn ROS2, hỗ trợ .db3 + .mcap
Phân tích	Polars, DuckDB	Đa luồng, xử lý hàng triệu dòng trong giây
Agent	LangGraph + LLM	Reasoning loop với tool calling
LLM	Claude/GPT API → vLLM	Phase 1: API nhanh, Phase 2: self-host
Đánh giá	DeepEval	LLM-as-judge cho chất lượng giải thích

Backend	FastAPI	Async, nhanh, phù hợp streaming
Frontend	React / Next.js	Dashboard timeline + chat
Deploy	Docker	Đóng gói toàn bộ pipeline

PRD

1. Kiến trúc hệ thống

Hệ thống gồm 4 tầng chính, xử lý tuần tự từ dữ liệu thô đến giao diện người dùng:

Tầng 0: Data Salvage

Nhiệm vụ: Xử lý các file .mcap hoặc .db3 bị hỏng cấu trúc do node crash hoặc mất nguồn đột ngột.

Hành động: Nếu công cụ đọc báo lỗi, một script Python chạy ngầm sẽ tự động quét ở mức nhị phân để khôi phục và đóng gói lại các message còn nguyên vẹn, đảm bảo luôn có dữ liệu đầu vào cho bước sau.

Tầng 1: Denormalization & Analytics (Polars)

Nhiệm vụ: Biến đổi dữ liệu dạng cây phức tạp của ROS2 thành dữ liệu phẳng (flatten) và tính toán các chỉ số cơ bản.

Hành động:

- Denormalizer: Viết các script bóc tách riêng từng loại dữ liệu (tọa độ x,y,theta từ Odometry, chuỗi frame_id từ TFMessage).
- Polars Analytics: Load dữ liệu đã bóc tách để tính toán các logic cứng (tần số rớt, độ lệch timestamp, giá trị NaN). Polars xử lý đa luồng giúp việc lướt qua hàng chục triệu dòng dữ liệu diễn ra chỉ trong vài giây.
- Scrubber: Lọc dữ liệu nhạy cảm (PII, GPS coordinates, credentials) trước khi gửi cho LLM.

Output: Anomaly JSON (vài KB) + In-Memory Polars DataFrames.

Tầng 2A: LLM Agent (LangGraph)

Nhiệm vụ: Đóng vai trò là "bộ não" phân tích nguyên nhân gốc rễ (Root Cause Analysis).

Hành động:

- Phase 1 (MVP): Sử dụng Claude/GPT API để validate ý tưởng nhanh.
- Phase 2 (Production): Host mô hình LLM bằng vLLM engine nếu cần self-host.
- Tool-Calling Loop: Agent nhận anomaly JSON → suy luận → gọi tool lấy thêm data → suy luận tiếp → đủ thông tin → sinh chẩn đoán.

Tầng 2B: Polars Tool Gateway

Nhiệm vụ: Cung cấp các tools cho agent gọi ngược Tầng 1 khi cần data chi tiết hơn.

Các tools:

- `check_tf_tree_gap(start_time, end_time)`: Kiểm tra TF tree có bị gãy trong khoảng thời gian.
- `query_frequency(topic, time_range)`: Lấy frequency thực tế của topic.
- `compare_topics(topic_a, topic_b)`: So sánh 2 topic có mất data cùng lúc không.
- `get_sample_messages(topic, time_sec, count)`: Lấy mẫu message quanh thời điểm lỗi.
- `search_known_issues(pattern)`: Tra cứu lỗi Nav2 phổ biến.

Mỗi tool trả kết quả qua Scrubber Gateway trước khi gửi về cho agent.

Tầng 3: Dashboard + HITL (FastAPI + React)

Nhiệm vụ: Trực quan hóa lỗi để kỹ sư quyết định.

Các thành phần:

- Timeline: Trực thời gian toàn bộ bag, mỗi topic là một hàng ngang, đánh dấu đỏ tại thời điểm lỗi.
- AI Explanation + Recommendation: Hiển thị giải thích nguyên nhân và đề xuất fix từ agent.
- Human-in-the-Loop: Kỹ sư duyệt (Approve) hoặc từ chối (Reject) từng chẩn đoán.
- Progressive Loading: Analyzer xong phần nào hiện phần đó, không chờ hết pipeline mới hiện.

2. Các loại lỗi cần phát hiện

Hệ thống tập trung phát hiện 6 loại lỗi phổ biến nhất của AMR chạy Nav2:

STT	Loại lỗi	Giải thích	Thuật ngữ
1	TF gap	Kill robot_state_publisher → TF tree mất liên kết, Nav2 không biết vị trí các bộ phận robot so với nhau → không thể lập kế hoạch đường đi	TF (Transform): hệ tọa độ mô tả vị trí/hướng giữa các bộ phận robot. TF tree: cây từ map → odom → base_link
2	Node crash (LiDAR)	Kill laser node → topic /scan chết hoàn toàn → robot mù về môi trường xung quanh, không tránh được vật cản	Node: tiến trình ROS2 độc lập. LiDAR: cảm biến quét laser 360° đo khoảng cách vật cản
3	Frequency drop	Throttle /scan xuống 1Hz (bình thường ~7Hz) → LiDAR vẫn sống nhưng dữ liệu quá thưa, Nav2 phản ứng chậm với vật cản mới	Frequency (Hz): số lần dữ liệu gửi mỗi giây. Throttle: giới hạn tần số publish
4	Timestamp drift	Publish IMU với timestamp tăng dần lệch → EKF tính sai → vị trí ước tính trôi dần khỏi thực tế, khó phát hiện bằng mắt	Timestamp: nhãn thời gian mỗi message. Drift: lệch dần theo thời gian. EKF: thuật toán kết hợp sensor
5	NaN data quality	Inject NaN vào IMU linear_acceleration → giá trị vô nghĩa lây lan qua mọi phép tính → EKF/fusion mất hoàn toàn khả năng định vị	NaN (Not a Number): giá trị đặc biệt khi phép tính không hợp lệ (0/0). Sensor fusion: kết hợp nhiều sensor
6	Topic dead (IMU)	Kill IMU node → /imu ngừng hoàn toàn → EKF mất dữ liệu gia tốc và góc quay → chỉ còn odometry bánh xe (kém chính xác)	IMU: cảm biến đo gia tốc + tốc độ góc. Odometry: ước tính vị trí từ vòng quay bánh xe

3. Dataset

3.1. Nguồn dữ liệu

Dữ liệu được thu từ Gazebo simulation với TurtleBot3 (model waffle) chạy Nav2 stack. Lý do chọn simulation: tự inject lỗi có chủ đích, tự có ground truth, tạo hàng loạt kịch bản tự động, không cần robot thật.

3.2. Các topic được record

Topic	Nhóm	Mục đích
/tf, /tf_static	Transform	TF tree — detect tf gap, broken chain map→odom→base_link
/scan	Sensor	LiDAR — detect frequency drop, topic dead
/odom	Robot State	Odometry — detect timestamp lệch, drift
/imu	Sensor	IMU — detect NaN, sensor freeze, clock drift
/cmd_vel	Command	Velocity command — detect node crash (velocity mất đột ngột)
/amcl_pose	State Estimate	Localization — detect mất định vị
/diagnostics	Health	System health — correlate failures với component health

Nguyên tắc chọn topic: Không record tất cả (~100 topic). Chỉ record 8 topic cốt lõi cover 4 nhóm chính (Sensor, State, Command, Transform, Health). Giảm dung lượng từ ~6.5GB/3 phút xuống ~5MB/44 giây.

3.3. Số lượng dataset

Loại	Số bag	Mục đích
Healthy (baseline)	10	Analyzer phải nói "không có lỗi"
Mỗi loại lỗi đơn (6 loại)	$5 \times 6 = 30$	Test detection từng loại lỗi
Lỗi kết hợp (2-3 lỗi cùng lúc)	10	Test correlation + root cause analysis
Tổng	~50 bag	Đủ cho MVP benchmark

TÀI LIỆU THAM KHẢO

Mentor Duty 1/8

Trong quá trình robot định vị, môi trường xung quanh không phải lúc nào cũng tĩnh. Khi có vật thể động (người đi qua, xe đẩy, hàng hóa được di chuyển) xuất hiện gần robot — đặc biệt gần LiDAR — sẽ gây ra các vấn đề:

- Mất tia (ray missing): vật thể chắn một phần tia laser → LiDAR không quét được vùng phía sau vật thể → dữ liệu /scan bị thiếu
- Đọc sai khoảng cách: LiDAR đo được vật thể động thay vì tường/vật cản cố định → AMCL so sánh với map tĩnh thấy không khớp → định vị bị nhiễu
- Định vị trôi: nếu vật thể động xuất hiện liên tục → AMCL liên tục bị nhiễu → particle cloud phân tán → robot không chắc chắn về vị trí của mình

Repo tham khảo

<https://github.com/vikramnagashoka/rosbag-resurrector>

